Revive

Dossier de lancement — M1 MIAGE Apprentissage — v0.1.0-SNAPSHOT

Jorys KANCEL, SALEKHA FALL, SARAH METREF, CAMILIA SABA, MOHAMED GUEYE · 20 octobre 2026

01 Équipe, comptes développeurs et dépôt

MEMBRE	COMPTE GITHUB	EMAIL DES COMMITS (`GIT CONFIG USER.EMAIL`)
Jorys KANCEL	`@Sinie-Kun`	jorys.kancel@gmail.com
SALEKHA FALL	`@salekhaf`	zulekhfall@gmail.com
SARAH METREF	`@SarahMETREF`	metrefsarah@gmail.com
CAMILIA SABA	`@fromwonderland`	camilia200@outlook.fr
MOHAMED GUEYE	`@Moh-svg04`	gueyemoe04@gmail.com

Dépôt : https://github.com/Moh-svg04/reUseAi-projet-m1-miage **page de projet** : https://Moh-svg04.github.io/reUseAi-projet-m1-miage/.

Projet et caractère innovant

Le problème. Les associations récupèrent des objets auprès des particuliers, mais organiser les collectes et redistribuer les objets peut être compliqué. Il faut choisir les adresses à visiter, organiser les tournées, respecter les capacités des véhicules et répartir les objets entre les associations selon leurs besoins.

Revive vise à faciliter cette organisation en regroupant ces différentes étapes dans une même application. L'objectif est de mieux organiser les collectes et de donner une seconde vie aux objets en les orientant vers les associations qui en ont besoin.

Ce qui est innovant :

- **L'optimisation plutôt que la simple gestion.** L'application ne sert pas seulement à enregistrer des objets et des adresses. Elle vise aussi à proposer des solutions : regrouper les collectes proches, organiser les tournées, préparer les chargements et répartir les objets selon les besoins des associations. Ces fonctionnalités pourront s'appuyer sur différents algorithmes d'optimisation.
- **L'utilisation du Machine Learning.** Nous souhaitons intégrer du Machine Learning pour estimer les besoins des associations selon les catégories d'objets. Par exemple, à partir des données disponibles, le modèle pourrait estimer les quantités de vêtements, de meubles ou de matériel informatique dont les associations pourraient avoir besoin. Ces estimations pourraient aider à mieux préparer la redistribution.
- **Des fonctionnalités liées entre elles.** Les différentes étapes se complètent : le regroupement des adresses facilite l'organisation des tournées, les capacités des véhicules permettent de préparer les chargements et les besoins des associations aident à répartir les objets collectés.

- **Une dimension solidaire et environnementale.** En facilitant la collecte et la redistribution, Revive cherche à favoriser le réemploi des objets et à limiter le gaspillage.

Ce qui existe déjà. Il existe des outils pour calculer des itinéraires, gérer des stocks ou organiser des opérations logistiques. Revive souhaite réunir ces besoins dans une même application et associer l'optimisation logistique à une estimation des besoins grâce au Machine Learning.

Scénarios d'usage

En tant que responsable de collecte, je peux enregistrer les objets à récupérer et leurs adresses afin de préparer une campagne de collecte.

En tant que responsable de collecte, je peux regrouper les adresses proches afin de faciliter l'organisation des déplacements.

En tant que responsable logistique, je peux préparer les chargements des véhicules en tenant compte de leurs limites de poids et de volume.

En tant que responsable logistique, je peux organiser les tournées afin de limiter les distances parcourues et de mieux planifier les collectes.

En tant que responsable d'association, je peux renseigner les besoins de mon association afin de faciliter la répartition des objets collectés.

En tant que responsable logistique, je peux comparer plusieurs propositions d'organisation afin de choisir celle qui répond le mieux aux contraintes de collecte et de redistribution.

En tant que responsable d'association, je pourrais consulter une estimation des besoins futurs par catégorie d'objets afin d'anticiper les quantités nécessaires, grâce au modèle de Machine Learning envisagé.

02 Plan des features

Une feature a un effet sur **la BD, le back, l'API REST et le front**. Chaque feature est portée par **une seule personne** et repose sur **un algorithme identifié**. La répartition ci-dessous est une proposition initiale tournante : elle devra être validée par l'équipe et chaque fiche devra préciser les contraintes, les cas limites et la complexité de l'algorithme réellement implémenté.

VERSION	MEMBRE 1	MEMBRE 2	MEMBRE 3	MEMBRE 4
v0.1.0 24/11	Feature — algorithme			
v0.2.0 08/12				
v1.0.0 19/01				

Les complexités seront documentées dans les fiches de features en fonction des structures de données et de l'implémentation choisies. Les fonctionnalités dépendant de données historiques seront évaluées avec prudence : les jeux synthétiques permettent de tester l'algorithme, mais ne suffisent pas à revendiquer une précision prédictive réelle. La

distinction entre fonctionnalités devra être justifiée par des objectifs et contraintes différents, notamment pour l'affectation initiale et la répartition globale, ainsi que pour la comparaison et la validation des plans.

03 Architecture

```
Front  --HTTP/JSON-->  API REST  -->  Back (cas d'utilisation)  --JPA-->  BD
(web)                (Spring MVC)      (Java 21, hexagonal)            (PostgreSQL)
```

COUCHE	CHOIX	REMARQUES
Front	application web : HTML, CSS, JavaScript natif	client de l'API REST uniquement ; dossier `front/`
API REST	Spring Boot 4 (Spring MVC), OpenAPI / Swagger UI	versionnée sous `/api/v1`, erreurs au format RFC 9457
Back	Java 21, Spring Boot 4, Spring Data JPA, Bean Validation	**architecture hexagonale**
BD	PostgreSQL 17 (H2 pour les tests)	schéma versionné par **Flyway**, une migration par feature
Qualité	JUnit 5, AssertJ, ArchUnit, JaCoCo, Codecov, SonarCloud	intégration continue GitHub Actions
Conteneurs	Docker Compose	BD dès v0.1.0, application en v0.2.0

Le modèle métier commun s'organise autour du lot d'objets, lié à une campagne, une localisation, un statut de collecte et, après récupération, à une affectation éventuelle. Les véhicules portent des capacités en poids et en volume ; les associations expriment des besoins par catégorie et quantité. Les migrations Flyway versionnent les évolutions du schéma.

Qualité du code. Au-delà de l'architecture hexagonale :

- **Clean Code** : noms métier explicites, fonctions courtes, pas de duplication, commentaires qui expliquent le pourquoi ;
- **SOLID**, objets valeur immuables et invariants vérifiés à la construction ;
- **design patterns** quand ils servent, nommés dans les fiches de features ;
- conventions **REST** et règles de sécurité **OWASP** ;
- quality gate SonarCloud au vert.

04 Démarche pour chaque feature

Les étapes s'enchaînent **dans cet ordre, sans en sauter** : on ne commence une étape que lorsque la précédente est terminée.

ÉTAPE	CE QU'ON FAIT	TERMINÉ QUAND
1. Idéation	Besoin et scénario ; **algorithme retenu**, complexité, alternatives ; impact sur le schéma et l'API ; critères d'acceptation. Issue + fiche `doc/features/`.	La fiche est validée par un autre membre.

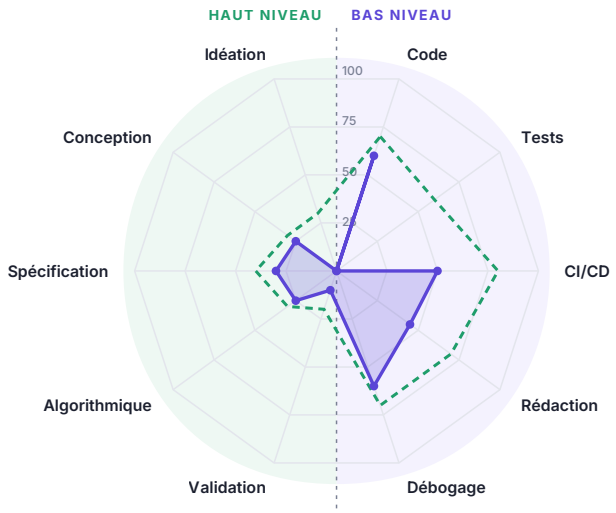
ÉTAPE	CE QU'ON FAIT	TERMINÉ QUAND
2. Back	Migration Flyway ; **domaine** en Java pur (modèle, règles, algorithme) ; **application** (ports, service) ; adaptateur de persistance.	Le code compile et ArchUnit passe.
3. Tests	Unitaires de l'algorithme (cas limites, invariants, propriétés) ; services avec faux adaptateurs ; intégration de la persistance.	`./gradlew build` passe ; chaque critère d'acceptation est couvert.
4. API REST	Endpoints `/api/v1`, DTO validés, erreurs `problem+json`, OpenAPI ; tests de l'API.	Visible dans Swagger UI ; tests de l'API verts.
5. Front	Écrans qui consomment **uniquement** l'API.	Scénario jouable de bout en bout ; PR relue ; fiche complétée avec son araignée.

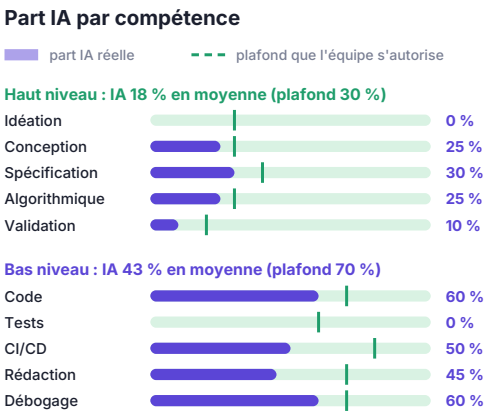
05 Protocole d'usage de l'IA

Principe : l'IA assiste, l'humain reste l'auteur. On ne laisse jamais l'IA tout faire sans comprendre : tout ce qui est comité a été relu et compris par son auteur, capable de l'expliquer, de le modifier et de le refaire sans aide. Les consignes des assistants sont ce dans `CLAUDE.md`.

ACTIVITÉ	CE QUE FAIT L'IA	CE QUE FAIT L'HUMAIN
Idéation et conception	Critique les idées, propose des options	Choisit, justifie, rédige la fiche
Algorithme	Explique, relit, signale les cas oubliés	**Conçoit et écrit le cœur de l'algorithme**
Code technique	Peut générer un premier jet	Relit chaque ligne, adapte, assume
Tests	Suggère des cas	Définit cas limites et propriétés ; vérifie qu'un test échoue quand on casse le code
CI/CD, conteneurisation	Peut générer workflows et Dockerfile	Comprend chaque étape, vérifie l'exécution, gère les secrets
Documentation	Aide à rédiger et mettre en forme	Valide le contenu, vérifie les faits

06 Annexe : part IA / part humaine





Valeurs dans `doc/part-ia.json`, régénérées à chaque compilation.